

The Key Part He Uses (The "Another Way"):

He creates a FormData object from the entire form:

JavaScript

```
const formData = new FormData(form); // 'form' is the <form> element
```

This automatically collects all selected radio buttons from the form. For each question (like q1, q2, etc.), it gives you:

- The name (e.g., "q1")
- The selected value (e.g., "Virat Kohli" if you chose that)

Then, he loops over all of them automatically (no need to write code for each question):

JavaScript

```
for (let [name, value] of formData.entries()) {  
  // 'name' is like "q1", "q2", etc.  
  // 'value' is what the user selected for that question  
  
  if (correctAnswers[name] === value) {  
    score++; // correct answer!  
  }  
}
```

This loop runs once for each question that has a selected answer — even if there are 5, 10, or 100 questions, you write the loop only once. No "100 times" code!

Full Simple Example Code (from the video, cleaned up):

JavaScript

```
const form = document.querySelector('form'); // select the form  
const output = document.getElementById('output'); // where to show score
```

```
// Correct answers (change these to match your quiz)
```

```
const correctAnswers = {  
  q1: "Virat Kohli", // example correct for question 1  
  q2: "West Indies",  
  q3: "Sachin Tendulkar",  
  q4: "264",  
  q5: "Shane Warne" // example  
};
```

```
form.addEventListener('submit', (e) => {  
  e.preventDefault(); // stop page refresh
```

```
  let score = 0;
```

```
  const formData = new FormData(form); // magic: gets all answers automatically
```

```
  for (let [name, value] of formData.entries()) {  
    // Check if user's answer matches the correct one  
    if (correctAnswers[name] === value) {
```

```

        score++;
    }
}

// Show the result
output.textContent = `Your score is ${score} out of 5`;

form.reset(); // clear the form for next try
});

```

Why This is Better (No "100 times"):

Without FormData + loop: You would have to manually do something like:

JavaScript

```

const q1Answer = document.querySelector('input[name="q1"]:checked').value;
const q2Answer = document.querySelector('input[name="q2"]:checked').value;

```

- // ... repeat for q3, q4, q5, q100 → horrible!
That's the bad way (repeating 100 times).
- With this way: Just one loop handles all questions automatically. Perfect for 5 or 100 questions.

This is what the educator means by using new FormData(form) — it's the clean, scalable way. Paste this code in your JS file and it should work exactly like in the video.

FormData in JavaScript is a built-in object (part of the Web API) that makes it super easy to collect and work with data from an HTML <form> — especially the values users have entered or selected (like text inputs, checkboxes, radio buttons, or even file uploads).

Main Purpose

- It creates a set of **key/value pairs** (like "q1" = "Virat Kohli") from your form fields.
- It's mostly used for sending form data to a server via fetch() or XMLHttpRequest (AJAX), without refreshing the page.
- It automatically handles the correct format (multipart/form-data), which is needed for forms with files or complex data.

How It's Created

You can make one like this:

JavaScript

```

const form = document.querySelector('form'); // grab your <form> element
const formData = new FormData(form); // boom! It auto-fills with all form data

```

(You can also create an empty one: new FormData() and add stuff manually with .append().)

Why It's Awesome (Especially in the Quiz Video)

In the cricket quiz, there are multiple questions (radio buttons with names like "q1", "q2", etc.).

Without FormData, you'd have to manually grab each one:

JavaScript

```
const q1 = document.querySelector('input[name="q1"]:checked').value;  
const q2 = document.querySelector('input[name="q2"]:checked').value;  
// ... repeat for every question → painful if 100 questions!
```

With FormData, it grabs **everything automatically**:

JavaScript

```
const formData = new FormData(form);  
  
for (let [name, value] of formData.entries()) {  
  console.log(name, value); // e.g., "q1" "Virat Kohli"  
  // Now check if value matches correct answer for that 'name'  
}
```

This loop runs for every question — no matter if there are 5 or 100. That's the "another way, not doing 100 times" the educator meant!

Quick Example

HTML form:

HTML

```
<form id="myForm">  
  <input name="username" value="Rohit">  
  <input name="age" value="25">  
  <button type="submit">Submit</button>  
</form>
```

JS:

JavaScript

```
const form = document.getElementById('myForm');  
form.addEventListener('submit', (e) => {  
  e.preventDefault();  
  const data = new FormData(form);  
  
  for (let [key, val] of data) {  
    console.log(key + ': ' + val); // username: Rohit age: 25  
  }  
});
```

FormData is a lifesaver for forms — clean, automatic, and scalable. If you're building quizzes, logins, uploads, etc., this is your go-to! 😊 Let me know if you want a full workin

FormData shines especially when uploading files (single or multiple), often combined with other form fields like text inputs. It handles the complex `multipart/form-data` format automatically, including file names, types, and boundaries.

Why FormData for File Uploads?

- Normal forms with `enctype="multipart/form-data"` submit files, but JavaScript needs **FormData** for AJAX uploads (no page refresh).
- It appends **File** or **Blob** objects directly.
- When sending via `fetch()` or `XMLHttpRequest`, **do not** set the `Content-Type` header — FormData adds it with the correct boundary.

1. Basic Single File Upload

HTML:

```
```html
<form id="uploadForm">
 <input type="text" name="username" placeholder="Your name">
 <input type="file" name="avatar" id="fileInput">
 <button type="submit">Upload</button>
</form>
```
```

JavaScript (using Fetch):

```
```javascript
const form = document.getElementById('uploadForm');

form.addEventListener('submit', async (e) => {
 e.preventDefault(); // Stop normal submit

 const formData = new FormData(form); // Automatically grabs text + file!

 try {
 const response = await fetch('/upload', {
 method: 'POST',
 body: formData // Send as-is
 });

 const result = await response.json();
 console.log('Success:', result);
 } catch (error) {
```

```

 console.error('Error:', error);
 }
});
...

```

- `new FormData(form)` collects everything: the text from "username" and the selected file from "avatar".

### ### 2. Multiple Files Upload

For multiple files, use the `multiple` attribute on the input.

HTML:

```

```html
<input type="file" name="photos" multiple accept="image/*">
...

```

JavaScript:

```

```javascript
const fileInput = document.querySelector('input[type="file"]');

fileInput.addEventListener('change', () => {
 const formData = new FormData();

 // Loop through all selected files
 for (let i = 0; i < fileInput.files.length; i++) {
 formData.append('photos', fileInput.files[i]); // Same key "photos" for all
 // Or unique: formData.append(`photos[${i}]`, fileInput.files[i]);
 }

 // Add other fields if needed
 formData.append('description', 'My vacation pics');

 fetch('/upload-multiple', {
 method: 'POST',
 body: formData
 });
});
...

```

- Server-side (e.g., PHP/Node), the files arrive as an array under the key "photos".

### ### 3. Manually Adding Files (Without a Form)

If no `` exists (e.g., drag-and-drop):

```
````javascript
const formData = new FormData();

// From an input
const fileInput = document.getElementById('myFile');
formData.append('myFile', fileInput.files[0]);

// Or from a Blob (e.g., canvas image)
const canvas = document.getElementById('myCanvas');
canvas.toBlob((blob) => {
  formData.append('canvasImage', blob, 'drawing.png');
});

// Send
fetch('/save', { method: 'POST', body: formData });
````
```

#### ### Key Tips

- Inspect contents: ``for (let [key, value] of formData.entries()) { console.log(key, value); }``
- Progress tracking: Use ``xhr.upload.onprogress`` (with XMLHttpRequest) or Fetch with a wrapper.
- Multiple files with same name: Server treats them as an array (e.g., ``photos[]`` in PHP).

FormData makes file uploads simple and reliable — no manual string building or headers needed. Check MDN docs for more:

<https://developer.mozilla.org/en-US/docs/Web/API/FormData> 😊 Let me know if you want a full working demo!